

Name: Frank Li

Date: Aug 23rd, 2026

Course: Python 100 Assignment 08_ Model 08

Assignment 08 - Python Programming Basics

Module 08 - Collections of data

Introduction

In this assignment, you will learn about how to create multi-module applications. Course assignments help you learn through reading, watching demonstrations, performing programming in Python, and reflecting on what you learned through writing.

This assignment includes the following tasks:

- Read module text.
- Watch the module videos.
- Create an application.
- Document your knowledge.
- Submit your work.

File Name:

- The file is named Assignment08.py

Script Header:

- The script headers are included in all files your name, the current date, and activity.

Constants:

- `FILE_NAME: str = "EmployeeRatings.json"`
- `MENU: str = ""`
---- Employee Ratings -----
Select from the following menu:
1. Show current employee rating data.
2. Enter new employee rating data.
3. Save data to a file.
4. Exit the program.
- -----

Variables:

- `employees: list = [] # a table of employees data`
- `menu_choice = ""`

Classes:

- The application includes a class named FileProcessor.
- The application includes a class named IO.
- The application includes a class named Person.
- The application includes a class named Employee.
- All classes include descriptive document strings.

Class Properties:

- The application includes properties for first_name: str and defaults to an empty string.
- The application includes properties for last_name: str and defaults to an empty string.
- The application includes properties for review_date: datetime.date and defaults to "1900-01-01".
- The application includes properties for review_rating: int and defaults to 3.
- The application's properties must include simple validation code.

Class Methods:

- The application includes a __str__() method to extract comma-separate data from its data objects.

Functions:

- All functions include descriptive document strings.
- All functions the include exception blocks use the output_error_messages() function for handling error messages.
- All non-instance functions use the @staticmethod decorator (The ones in the FileProcessor and IO classes.)
- The application includes functions with the following names and parameters: output_error_messages(message: str, error: Exception = None)
- output_menu(menu: str)
- input_menu_choice()
- output_employee_data(employee_data: list)
- input_employee_data(employee_data: list, employee_type: object)
- read_employee_data_from_file(file_name: str, employee_data: list, employee_type: object)
- write_employee_data_to_file(file_name: str, employee_data: list):

Input / Output:

- On menu choice 1, the presents the collected data. (Note this is a different order that we used previously!)
- On menu choice 2, the application prompts the user to enter the employee's first name and last name, followed by the review date and rating.
- Data collected for menu choice 2 is added to a two-dimensional list table (list of Employee objects).

Summary: Please refer to the screenshots

1, run the script to display the menu; try wrong input to show the error handling for wrong values

```
Enter your menu choice number: a
There was a problem with your menu choice.

-- Technical Error Message --
Please choose only 1, 2, 3, or 4.
Inappropriate argument value (of correct type).
<class 'ValueError'>
```

2, choose 1 and input wrong first, last name, review date and rating to test the error handling

```
Enter your menu choice number: 2
What is the employee's first name? @! Y#@!
That value is not the correct type of data.

-- Technical Error Message --
The first name should contain only letters and numbers.
Inappropriate argument value (of correct type).
<class 'ValueError'>
```

```
Enter your menu choice number: 2
What is the employee's first name? Frank002
What is the employee's last name? Li!@#$
That value is not the correct type of data.

-- Technical Error Message --
The last name should contain only letters and numbers.
Inappropriate argument value (of correct type).
<class 'ValueError'>
```

```
Enter your menu choice number: 2
What is the employee's first name? Frank002
What is the employee's last name? Li002
What is their review date (YYYY-MM-DD)? asdf-dd-aa
That value is not the correct type of data.

-- Technical Error Message --
Incorrect data format, should be YYYY-MM-DD
Inappropriate argument value (of correct type).
<class 'ValueError'>
```

```
Enter your menu choice number: 2
What is the employee's first name? Frank002
What is the employee's last name? Li002
What is their review date (YYYY-MM-DD)? 2026-08-23
What is their review rating (1-5)? d
That value is not the correct type of data.

-- Technical Error Message --
invalid literal for int() with base 10: 'd'
Inappropriate argument value (of correct type).
<class 'ValueError'>
```

3, choose 2 and input correct values; and then choose 1 to show the inputs

```
-----
Enter your menu choice number: 2
What is the employee's first name? Frank002
What is the employee's last name? Li002
What is their review date (YYYY-MM-DD)? 2026-08-23
What is their review rating (1-5)? 5

Employee review data was added:
Frank002,Li002,2026-08-23,5
```

```
---- Employee Ratings -----
Select from the following menu:
  1. Show current employee rating data.
  2. Enter new employee rating data.
  3. Save data to a file.
  4. Exit the program.
-----

Enter your menu choice number: 1

-----
John Smith is rated as 5 (Leading) on 2026-08-19
Mary Jones is rated as 4 (Strong) on 2026-08-20
Frank001 0001Li is rated as 5 (Leading) on 2028-08-20
Frank002 Li002 is rated as 5 (Leading) on 2026-08-23
-----
```

4, choose 3 to save the inputs into Jason file



```
[
  {
    "FirstName": "John",
    "LastName": "Smith",
    "ReviewDate": "2026-08-19",
    "ReviewRating": 5
  },
  {
    "FirstName": "Mary",
    "LastName": "Jones",
    "ReviewDate": "2026-08-20",
    "ReviewRating": 4
  },
  {
    "FirstName": "Frank001",
    "LastName": "0001Li",
    "ReviewDate": "2028-08-20",
    "ReviewRating": 5
  },
  {
    "FirstName": "Frank002",
    "LastName": "Li002",
    "ReviewDate": "2026-08-23",
    "ReviewRating": 5
  }
]
```

Result:

- The application takes the user's input for an employee's first, last name, review date and rating.
- The application validates the user's input for an employee's first, last name, review date and rating.
- The application displays the user's input for employee's first, last name, review date and rating.
- The application saves the application's data into "EmployeeRatings.json" as valid JSON syntax.
- The application allows users to enter multiple employee reviews.
- The application allows users to display multiple employee reviews.
- The application allows users to save multiple employee reviews to a file.
- The application runs correctly in both **PyCharm** and from the **console or terminal**.

The script can handle the errors below:

- The application provides structured error handling when the file is read into the list of employee rows.
- The application provides structured error handling when the user enters a first name.
- The application provides structured error handling when the user enters a last name.
- The application provides structured error handling when the user enters a review date (YYYY-MM-DD).
- The application provides structured error handling when the user enters a review rating (1,2,3,4, or 5).
- The application provides structured error handling when the data is written to the file.